Services in ASP.NET Core Web API

Back to ASP.NET Core Web API Tutorial

Services in ASP.NET Core Web API

In ASP.NET Core Web API, controllers are responsible for handling HTTP requests and returning responses. While it is possible to include business logic and data access logic directly in the controller, this approach violates the principle of separation of concerns and makes the code difficult to test and maintain.

Services in ASP.NET Core Web API are classes that encapsulate business logic and data access logic. They provide a clean separation between the controller and the underlying business logic, making the code easier to maintain, test, and reuse. Services can be injected into controllers using dependency injection, promoting loose coupling and testability.

In this tutorial, we will create a simple ASP.NET Core Web API project that manages a list of products. We will create a service to handle the business logic for managing products, and we will demonstrate how to register the service in the dependency injection container and inject it into the controller.

Why Services Solve These Problems

- **Separation of Logic:** Business rules and data access logic live in dedicated classes.
- **Testability:** Services can be unit tested independently of the controller.
- **Reusability:** Services can be shared between multiple controllers or other parts of the application.
- **Maintainability:** Changes to business logic only require modifications in the service layer.

Real-time Analogy: Think of a restaurant kitchen. The waiter (controller) takes orders and delivers food to the customer, but the waiter doesn't know how to cook. The kitchen (service) handles the actual cooking.

Where Should We Write Validation Logic?

Validation in ASP.NET Core Web API can be handled in two ways:

- **In the Controller:** Using `ModelState.IsValid` and `[ApiController]` attribute. Simple but mixes HTTP with business concerns.
- **In the Service:** Throwing custom exceptions or returning `null` when `SingleOrDefault()` finds no matching entity.

Real-time Analogy: Think of a kitchen where customers (or waiters) ask for "a pizza" or "a salad." The kitchen knows what it can make, but it doesn't care about the specific order details.

Step 1: Define an Interface (Contracts)

Create an interface named `IProductService` in `Models` folder. It will copy paste the following code.

```
namespace ProductServices.Models
{
    public interface IProductService
    {
        Task<IEnumerable<Product>> GetAllProducts();
        Task<Product> GetProductById(int id);
        Task<Product> CreateProduct(Product product);
        Task<bool> UpdateProduct(int id, Product product);
        Task<bool> DeleteProduct(int id);
    }
}
```

Real-time Analogy: Imagine a restaurant where waiters (controllers) only know what orders they can take and what food they can deliver. The kitchen (service) knows how to actually cook the ingredients.

Why Use Interfaces?

- **Dependency Injection:** Allows the framework to inject different implementations at runtime.
- **Testability:** Easy to create mock implementations for unit testing.
- **Flexibility:** Can swap implementations without changing the controller.

Real-time Analogy: If the kitchen is the chef, the waiter is the **interface** which clearly lists what they can do.

Step 2: Register the Service in the Program.cs

Next, please add the following code in `Program.cs` file.

```
builder.Services.AddScoped<IProductService, ProductService>();
```

Code Explanations

- ASP.NET Core uses **Dependency Injection** (DI) to inject
- Here, we register `IProductService` with its implementation `ProductService`.
- `AddScoped()` means a **new instance** of `ProductService` will be created for each HTTP request.

Real-time Analogy: This is like telling the **Restaurant Manager**: "Whenever a customer orders something from the menu, (`IProductService`), send it to the specific kitchen (`ProductService`)."

Step 3: Use the Service in the Controller

Finally, update the `ProductsController` as follows.

```
namespace ProductServices.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductsController : ControllerBase
    {
        private readonly IProductService _productService;

        public ProductsController(IProductService productService)
        {
            _productService = productService;
        }

        [HttpGet]
        public ActionResult<IEnumerable<Product>> GetProducts()
        {
            var products = _productService.GetAllProducts();
            return Ok(products);
        }

        [HttpGet("{id}")]
        public ActionResult<Product> GetProduct(int id)
        {
            var product = _productService.GetProductById(id);
            if (product == null)
            {
                return NotFound();
            }
            return Ok(product);
        }

        [HttpPost]
        public ActionResult<Product> CreateProduct(Product product)
        {
            var newProduct = _productService.CreateProduct(product);
            return CreatedAtAction(nameof(GetProduct), new { id = newProduct.Id }, newProduct);
        }

        [HttpPut("{id}")]
        public ActionResult UpdateProduct(int id, [FromBody] Product updatedProduct)
        {
            var isUpdated = _productService.UpdateProduct(id, updatedProduct);
            if (!isUpdated)
            {
                return NotFound();
            }
            return NoContent();
        }

        [HttpDelete("{id}")]
        public ActionResult DeleteProduct(int id)
        {
            var isDeleted = _productService.DeleteProduct(id);
            if (!isDeleted)
            {
                return NotFound();
            }
            return NoContent();
        }
    }
}
```

Code Explanations

- This class provides the **Actual Implementation** of the Services.
- It encapsulates the **business logic** (e.g., creating, updating, and deleting products).
- It stores **Data Persisting** right here. Normally this data, database logic.
- The `Id` property is set to `products.Count + 1` to simulate auto-increment, only the service changes, not the controller.

Real-time Analogy: If the chef is the **Kitchen**, this service is the **Kitchen** which knows exactly how to cook.

Step 4: Test the Services

Now, we are ready to test the services. Please, run the application and navigate to `/swagger`.

Action Services JustRequestProductServices, ProductServicesUsing

Code Explanations

- ASP.NET Core uses **Dependency Injection** (DI) to inject
- Here, we inject `IProductServices` into `ProductServices`. `ProductServices` is hard-coded to **Data in Memory** (`ProductServices` will be created for each HTTP request)
- This provides **Testability** and **Flexibility**. We can easily swap implementations.

Real-time Analogy: This is like telling the **Restaurant Manager**: "Whenever a customer orders something from the menu, (`IProductServices`), send it to the specific kitchen (`ProductServices`)."

Code Explanations

- This code provides **Actually** `IProductServices`, not directly on `ProductServices`.
- ASP.NET Core's DI system automatically provides an instance of `ProductServices` when the controller is created.
- The controller doesn't know or care:
  - The actual implementation (`ProductServices`)
  - Only it provides between HTTP requests and the service methods.

Real-time Analogy: The **waiter (controller)** knows what food he has, but he does not cook and passes it to the **chef (service)**. Also brings the result back to the customer.

Services in ASP.NET Core Web API provide a clean and maintainable way to separate business logic from controllers. Instead of overloading controllers with data access logic, we can delegate responsibilities to services. Combined with dependency injection, services promote loose coupling, testability, and code reuse. The framework's built-in DI container makes it easy to manage service lifetimes (scoped, singleton, transient) and inject dependencies into controllers. This approach ensures **scalability, maintainability, testability, and reduced architecture**, making your API robust and production-ready.

Dot Net Tutorials
About Author: Pranaya Rout
Pranaya Rout has published more than 3000 articles in his 11 years long career. Pranaya Rout has good experience with Microsoft technologies, including C#, VB.NET, ASP.NET Web Forms, MVC, Web API, SQL Server, HTML, CSS, JQuery, JSON, Angular, Bootstrap, Azure, Entity Framework, Linq, Razor, Visual Studio, Microsoft Azure, Blockchain, Machine Learning, Artificial Intelligence, Cloud Computing